

## **Bài thực hành số 2: Sử dụng Web3.py để tương tác với hợp đồng thông minh trên Ethereum**

### **1. Mục tiêu**

Bài thực hành nhằm giúp sinh viên nắm vững các kiến thức và thành thạo trong việc tương tác cấp thấp với Blockchain, từ đó hiểu rõ cách thức hoạt động của nền tảng này ở mức cơ bản. Đồng thời, sinh viên sẽ được rèn luyện kỹ năng quản lý mạng lưới phát triển và thực hiện các giao dịch cơ bản trên Blockchain. Bên cạnh đó, bài thực hành còn hướng đến việc nâng cao hiệu suất phát triển thông qua việc sử dụng Ape Framework, giúp sinh viên vận dụng hiệu quả các công cụ hỗ trợ. Sau khi hoàn thành, sinh viên có thể:

- Biết cách cài đặt và sử dụng thư viện Web3.py để kết nối với các node Ethereum.
- Hiểu và sử dụng các blockchain phát triển cá nhân như Geth để mô phỏng môi trường Ethereum cục bộ.
- Có khả năng viết script Python để thực hiện các giao dịch trên blockchain, bao gồm chuyển ETH và tương tác với hợp đồng thông minh.
- Làm quen và áp dụng Ape Framework như một công cụ toàn diện giúp hợp lý hóa quy trình phát triển, biên dịch, thử nghiệm và triển khai hợp đồng thông minh.
- Áp dụng các kỹ năng Vyper và Ape Framework để xây dựng một ứng dụng phi tập trung có tính thực tiễn cao, cụ thể là ứng dụng bỏ phiếu phi tập trung.
- Phân tích và chứng minh các giá trị mà tính chất phi tập trung (như sự minh bạch và an toàn khỏi gian lận) mang lại cho ứng dụng đã xây dựng.

### **2. Thời gian làm bài**

Dự kiến: 3.5 giờ.

### **3. Cơ sở lý thuyết và bài tập tại lớp**

#### **A. Sử dụng Web3.py để Tương tác với Hợp đồng Thông minh**

Phần này tập trung vào việc tương tác với blockchain và các hợp đồng đã được triển khai thông qua thư viện web3.py:

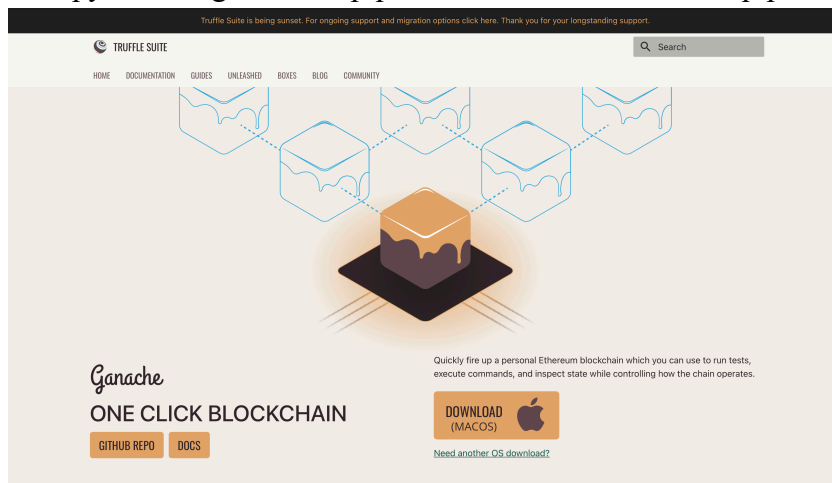
- Kết nối Node: Để tương tác với blockchain, cần khởi tạo đối tượng kết nối bằng cách cung cấp URL RPC (Remote Procedure Call) của node Ethereum. Các công cụ phát triển như Ganache cung cấp giao diện đồ họa và RPC server (ví dụ: `http://127.0.0.1:7545`), trong khi Geth có thể cung cấp các endpoint HTTP hoặc IPC (Inter-Process Communication) (ví dụ: `http://127.0.0.1:8545`).
- Thực hiện Giao dịch (Sending ETH): Một giao dịch chuyển ETH yêu cầu các thông số cơ bản như địa chỉ người gửi (from), địa chỉ người nhận (to), số lượng ETH (value) (thường được chuyển đổi sang đơn vị wei bằng `Web3.to_wei`),

nonce (để chống giao dịch lặp lại), và phí giao dịch (thông qua gas và gasPrice). Giao dịch phải được ký bằng khóa riêng tư của người gửi trước khi được gửi lên blockchain.

- Tương tác với Hợp đồng Thông minh:
  - Cần ABI (Application Binary Interface) và bytecode của hợp đồng để triển khai. ABI mô tả các hàm và sự kiện của hợp đồng.
  - Gọi hàm đọc trạng thái (Read/View functions): Các hàm không thay đổi trạng thái blockchain được thực thi bằng `.call()` và không tốn phí gas hay cần chữ ký.
  - Gọi hàm thay đổi trạng thái (Write/State-altering functions): Các hàm này cần được xây dựng thành giao dịch bằng `.build_transaction()`, yêu cầu chữ ký và phí gas. Đối với các hàm `@payable`, cần thêm trường 'value' vào đối tượng giao dịch để gửi ETH kèm theo.
- Truy vấn Sự kiện (Events): Có thể truy vấn các sự kiện đã được ghi lại (log) trên blockchain sau khi một hàm được thực thi, sử dụng `.get_logs()`.

### i. Thiết lập môi trường

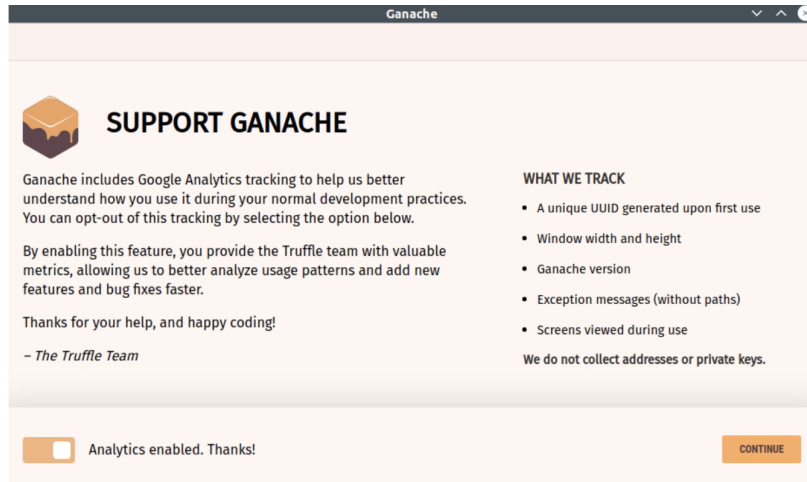
Cài đặt Web3.py. Sử dụng câu lệnh pip thực hiện cài đặt thư viện: `pip install web3`



Hình 1. Trang web Ganache

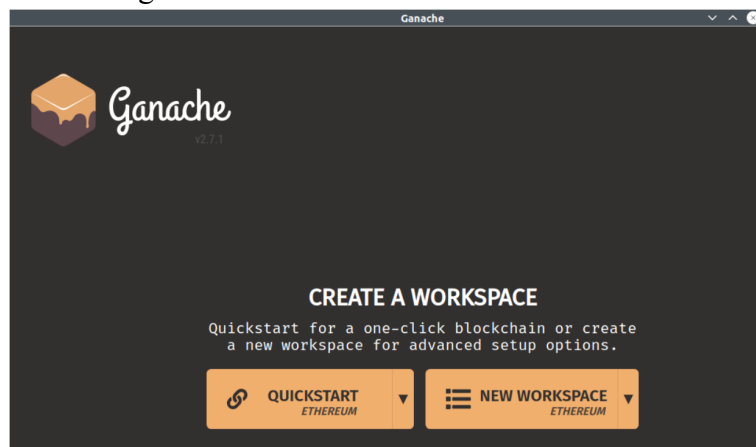
Ganache là một blockchain Ethereum riêng tư. Bằng cách sử dụng ứng dụng này, chúng ta có thể dễ dàng thử nghiệm và tương tác với blockchain. Một trong những tính năng độc đáo của ứng dụng này là nó có giao diện đồ họa trực quan. Trong khi đó, các nền tảng blockchain khác thường chỉ có giao diện dòng lệnh ví dụ như Geth.

Sử dụng Ganache bằng cách truy cập vào trang web, hỗ trợ trên môi trường Linux, Window, Mac: <https://archive.trufflesuite.com/ganache/>



Hình 2. Màn hình giới thiệu Ganache

Trong màn hình số 2, khi các em click continue thì ra màn hình số 3. Màn hình này là nơi các em bắt đầu tạo không gian làm việc (workspace). Trong Ganache, một workspace là một không gian cho một chuỗi khối. Thông thường, chúng ta sẽ tạo 2 không gian làm việc một không gian cho mục đích phát triển chương trình, một không gian cho mục đích kiểm thử chức năng.



Hình 3. Màn hình quickstart của Ganache.

Khi bạn chọn button New Workspace trong màn hình số 3, các em sẽ thấy màn hình số 4. Trong màn hình số 4, các em được yêu cầu đặt tên workspace, sau khi gõ tên vào khung workspace name thì các em sẽ bấm nút Start để bắt đầu. **Ngoài ra, các em sẽ được yêu cầu tìm hiểu các tab Server, Accounts & keys, Chain, Advance, About (Các em sẽ chụp hình các tab này và giới thiệu các mục có trong các tab này đưa vào file báo cáo nộp bài làm tại lớp).** Sau khi bấm nút Start, Hình số 5 xuất hiện cho phép quản lý các tài khoản trong mạng.

WORKSPACE

SERVER

ACCOUNTS & KEYS

CHAIN

ADVANCED

ABOUT

CANCEL

START

WORKSPACE

WORKSPACE NAME

fanatical-cat

A friendly name for this workspace.

TRUFFLE PROJECTS

Link Truffle projects to this workspace by adding their truffle-config.js or truffle.js file to this workspace.

This will show useful contract and event data to better understand what's going on under the hood.

ADD PROJECT

REMOVE PROJECT

Hình 4. Màn hình Workspace

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK

GAS PRICE

GAS LIMIT

HARDFORK

NETWORK ID

RPC SERVER

MINING STATUS

WORKSPACE

SWITCH

0

2000000000

6721975

MERGE

5777

HTTP://127.0.0.1:7545

AUTOMINING

IS335-BC

mnemonic

merry before later deal creek ahead logic false else wall warm piece

hd path

m/44'/60'/0'/0/account\_index

|                                            |            |          |       |  |
|--------------------------------------------|------------|----------|-------|--|
| ADDRESS                                    | BALANCE    | TX COUNT | INDEX |  |
| 0xbAdB559dfc3395c909fDB00557059009f1A276B8 | 100.00 ETH | 0        | 0     |  |
| 0x4d90214229c08D155DF8060D67f39B33b1De5424 | 100.00 ETH | 0        | 1     |  |
| 0x2aE9D70d93f5625347636676260f99aa7B281f7b | 100.00 ETH | 0        | 2     |  |
| 0xC02D91225940F92Ef953b0b6f4aFC7933d45abAB | 100.00 ETH | 0        | 3     |  |
| 0xD11D3f45a3A64684317404426c3151f51791e9a  | 100.00 ETH | 0        | 4     |  |
| 0x11e37dcde1b869203D3ecf5DeA62d4373458C0de | 100.00 ETH | 0        | 5     |  |
| 0x1273F45b92e082596435E68B20A8eE8523d34a25 | 100.00 ETH | 0        | 6     |  |
| 0xFebA9B8C046a38dd275E5E189FA160A0322A74eE | 100.00 ETH | 0        | 7     |  |
| 0x90234917819151ae00D23766BcF3028717543f75 | 100.00 ETH | 0        | 8     |  |

Hình 5. Màn hình quản lý tài khoản

## ii. Viết chương trình python tương tác với Ganache.

Trên màn hình số 5, mỗi tài khoản được cấp 100 ETH. Không gian làm việc được đặt tên là IS335-BC. Khối hiện tại là 0, có nghĩa là đây là khởi đầu của lịch sử chuỗi khối này. Máy chủ RPC nằm tại `http://127.0.0.1/7545`. Đây là một thông tin quan trọng. Khi các em muốn kết nối với chuỗi khối, các em cần tìm nơi để kết nối. Máy chủ RPC là một cách để kết nối với chuỗi khối. Trong màn hình này, các em nhìn vào tài khoản thứ ba từ trên xuống. Địa chỉ là `0x2aE9D70d93f5625347636676260f99aa7B281f7b`. Chúng ta sẽ sử dụng địa chỉ này trong một tập lệnh. Tuy nhiên, địa chỉ trên máy của các em có thể khác. Các địa chỉ được tạo trong Ganache theo một cách ngẫu nhiên. Vì vậy, các em cần điều chỉnh cho phù hợp trong đoạn chương trình của các em. Bây giờ ta viết một chương trình kết nối với đoạn chuỗi khối này, bằng cách tạo tập tin `hello_web3.py` với nội dung như hình số 5. Sau khi thực thi xong em chụp hình kết quả và đưa vào báo cáo nộp bài tập tại lớp. Lưu ý: 1ETH = 1,000,000,000,000,000 wei.

```

1 from web3 import Web3
2 w3 = Web3(Web3.HTTPProvider('http://127.0.0.1:7545'))
3 print(f"Python script is connected? {w3.is_connected()}")
4 # The balance of the third account. Change it according to generated addresses on your Ganache.
5 account_balance = w3.eth.get_balance('0x2aE9D70d93f5625347636676260f99aa7B281f7b')
6 print(f"The balance of the 3rd account is {account_balance}")

```

Hình 6. Đọc tài khoản thông tin từ chuỗi khối

Các em muốn biết khoá của từng khối thì các em có thể click vào khoá trên màn hình số 5, lúc này sẽ hiển thị 1 màn hình dialog, các em sẽ được cung cấp khoá riêng tư. Các em chụp hình khoá của tài khoản số 3 và nộp vào file báo cáo bài học tại lớp.

Ví dụ: Bây giờ các em sẽ viết đoạn chương trình chuyển 5 wei từ tài khoản số 3 xuống tài khoản số 6. Các em được cung cấp đoạn chương trình như hình số 7, nhiệm vụ các em là đọc hiểu ghi lại các công đoạn thực hiện của chương trình, đồng thời chụp hình để cho thấy thông tin tài khoản số 3 và số 6 trước và sau khi thực hiện chuyển đưa vào báo cáo để nộp bài tập tại lớp. Chú ý: các em sẽ tìm hiểu kỹ các thông tin transaction, đặc biệt là gas & gasPrice

```

from web3 import Web3
w3 = Web3(Web3.HTTPProvider('http://127.0.0.1:7545'))
# account & private key - third account
sender_private_key = "0x4d6af1ea397ee635291173296a47efc1237c55745dcd9654abdb86528991df9f"
sender_address = "0x2aE9D70d93f5625347636676260f99aa7B281f7b"
# sixth account
recipient_address = "0x11e37dcde1b869203D3ecf5DeA62d4373458C0de"
amount = Web3.to_wei(5, 'ether')

transaction = {
    'from': sender_address,
    'to': recipient_address,
    'value': amount,
    'gas': 21000,
    'gasPrice': Web3.to_wei('50', 'gwei'),
    'nonce': w3.eth.get_transaction_count(sender_address),
}

signed_transaction = w3.eth.account.sign_transaction(transaction, sender_private_key)
transaction_hash = w3.eth.send_raw_transaction(signed_transaction.raw_transaction)
print(transaction_hash)

```

Hình 7. Đoạn chương trình chuyển 5 wei từ tài khoản số 3 sang tài khoản số 6.

## B. Ape framework

### i. Thiết lập môi trường

Ape framework là một ứng dụng Python, do đó các em có thể cài đặt bằng câu lệnh pip (lưu ý: các em có thể được yêu cầu cài đặt thêm các thư viện để hỗ trợ cài đặt Ape framework).

Câu lệnh cài đặt: `pip install eth-ape`

Kiểm tra version ape: `ape --version`

Để bắt đầu tạo 1 project bằng câu lệnh: `ape init`

Sau đó các em nhập tên project và kiểm tra cấu trúc project không những file gì các em nhớ viết tên các file, thư mục trong báo cáo để nộp bài tập tại lớp.

## ii. Tạo 1 smart contract

Các em sử dụng ngôn ngữ lập trình Vyper đã được hướng dẫn trong bài lab 1 để tạo ra một smart contract như hình 8a, các em đặt tên file SimpleStorage.vy lưu trong thư mục scripts. Sử dụng câu lệnh `ape compile` để biên dịch (Lưu ý các em cần kiểm tra đã có Vyper trong plugin của Ape chưa bằng câu lệnh `ape plugins list`. Nếu chưa có Vyper thì các em buộc phải cài Vyper cho ape bằng câu lệnh: `ape plugins install vyper`). Nếu biên dịch thành công các em sẽ thấy như hình 8b.

```
# @version ^0.4.3

num: uint256
@external
def store(num: uint256):
    self.num = num

@external
def retrieve() -> uint256:
    return self.num
```

a. Smart contract đơn giản

```
INFO: Compiling using Vyper compiler '0.4.3'.
Input:
contracts/SimpleStorage.vy
SUCCESS: 'local project' compiled.
```

b. Kết quả biên dịch thành công smart contract trong ape framework.

Hình 8. Chương trình smart contract đơn giản

## iii. Kiểm thử smart contract

Trước khi kiểm thử, các em cần deploy smart contract này lên mạng chuỗi khối Ganache. Các em cần tạo 1 account alias kết hợp với 1 tài khoản trong mạng Ganache, ở đây ta sẽ tạo 1 account alias đặt tên là `is355` kết nối với tài khoản số 4 trong mạng Ganache các em sẽ thiết lập như hình 9. Các em lấy thông tin private key trên mạng Ganache, lưu ý là ở đây mình đang sử dụng tài khoản số 4, do đó các em lấy thông tin khoá riêng tư ở tài khoản số 4. Thông tin tài khoản Passphrase các em tự đặt, và lưu ý các em sẽ nhớ tài khoản này để chuẩn bị thực hiện cho quá trình deploy.

```
hatrung@Has-MacBook-Pro Bai02 % ape accounts import dev1
Enter Private Key:
Create Passphrase to encrypt account:
Repeat for confirmation:
SUCCESS: A new account '0x2aE9D70d93f5625347636676260f99aa7B281f7b' has been added with the id 'dev1'
```

Hình 9. Tạo tài khoản alias

Lúc này các em sẽ gặp một vấn đề trong quá trình tương tác với smart contract. Các em có thể compile và deploy được nhưng lúc này các em sẽ được báo lỗi `VM Exception while processing transaction: invalid opcode`. Các em tìm hiểu lỗi này và ghi trong báo cáo nộp bài tập tại lớp.

Một blockchain cá nhân khác các em có thể sử dụng để deploy smart contract và có thể tương tác phù hợp đó là Geth.

Để cài đặt Geth:

- Đối với môi trường Window, các em truy cập trang web và tải file .exe để về cài đặt: <https://geth.ethereum.org/downloads>. Tương tự cho môi trường Linux.
- Với môi trường Mac OS, các em phải cài đặt brew trước, sau đó thiết lập biến môi trường (sẽ có 2 trường hợp cho CPU: Apple Silicon và Intel; trong bài thực hành này thầy hướng dẫn cách cài đặt trên CPU Apple Silicon).
  - Tải brew về máy: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
  - Thêm homebrew vào Path: `echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile | eval "$(/opt/homebrew/bin/brew shellenv)"`
  - Kiểm tra cài đặt: `brew --version`
  - Thực hiện cài đặt Geth: `brew tap ethereum/ethereum | brew install ethereum`

Kiểm tra việc cài đặt Geth thành công bằng câu lệnh: `geth version`

Để chạy chuỗi khối Geth ta thực hiện lệnh sau: `geth --dev --http --http.api eth,web3,net --ipcpath /tmp/geth.ipc`. Lưu ý lệnh này chạy thì các em mở một terminal để thực hiện, cửa sổ này ghi lại toàn bộ log các em thao tác trên chuỗi khối Geth. Sau khi chạy lệnh này xong, các em chụp lại kết quả chạy bỏ vào file báo cáo. Các em nhớ khoang tròn port chạy Geth.

Lúc này, các em có thể kết nối với chuỗi khối thông qua HTTP hoặc IPC (inter-process communication) và WebSockets. Các em có thể chạy lệnh sau để kết nối với chuỗi khối thông qua việc sử dụng IPC: `geth attach /tmp/geth.ipc`

Các em có thể lấy địa chỉ của tài khoản đầu tiên trong chuỗi khối thông qua lệnh sau: `eth.accounts`. Các em chụp lại địa chỉ và bỏ vào file báo cáo.

Để kiểm tra balance của tài khoản các em có thể thực hiện câu lệnh sau: `web3.fromWei(eth.getBalance(eth.accounts[0]))`. Các em chụp lại kết quả balance của tài khoản bỏ vào file báo cáo.

Bây giờ ta tạo 1 tài khoản khác. Đầu tiên các em tạo thư mục `my_keystore`. Sau đó thực hiện câu lệnh tạo tài khoản mới lưu vào trong thư mục `my_keystore`, bằng câu lệnh sau: `clef newaccount --keystore my_keystore`.

Các em làm theo hướng dẫn và đặt password (Lưu ý các em phải nhớ password đã đặt để thực hiện việc kết nối với chuỗi khối Geth). Sau khi thực hiện xong, các em ghi lại thông tin địa chỉ của tài khoản vừa mới được tạo. Trên thư mục `my_keystore` lúc này tạo ra 1 file mới các em ghi tên file đó vào trong báo cáo để nộp bài tập tại lớp.

Sau đó các em chuyển tiền từ tài khoản ban đầu đến tài khoản mới vừa được tạo, thông qua lệnh sau: `eth.sendTransaction({from: eth.accounts[0], to:"0x95A6BBb44724F9d8b8C34F47503CDee860eC6b77", value: web3.toWei(10, "ether")})`. `0x95A6BBb44724F9d8b8C34F47503CDee860eC6b77` chính là địa chỉ của tài khoản mới được tạo ra (trên máy các em sinh ra địa chỉ nào thì thay thế địa chỉ đó vào). Sau khi thực hiện xong nhớ chụp lại kết quả đưa vào báo cáo.

Thực hiệ kiểm tra balance của tài khoản mới: `web3.fromWei(eth.getBalance("0x95A6BBb44724F9d8b8C34F47503CDee860eC6b77"))`.

Sau khi thực hiện các lệnh command line để thao tác kiểm tra và chuyển tiền từ các tài khoản khác nhau, bây giờ các em có thể thực hiện viết 1 chương trình python để tương

tác với chuỗi khối Geth. Các em tạo file `send_eth_on_geth.py` có nội dung như hình 10. Biến password ở đây là biến mà các em đã tạo password trước đó. Các em chạy xong cho biết phí và số tiền đã chuyển giữa 2 tài khoản này.

```
from web3 import Web3
w3 = Web3(Web3.IPCProvider('/tmp/geth.ipc'))
with open("my_keystore/UTC--2025-09-25T14-11-51.070973000Z--95a6bbb44724f9d8b8c34f47503cdee860ec6b77") as keyfile:
    encrypted_key = keyfile.read()
    password = "1234567890"
    sender_private_key = w3.eth.account.decrypt(encrypted_key, password)
    sender_address = "0x95A6BBb44724F9d8b8C34F47503CDee860eC6b77"
    recipient_address = Web3.to_checksum_address("0x71562b71999873db5b286df957af199ec94617f7")
    amount = Web3.to_wei(2, 'ether')
    transaction = {
        'chainId': 1337,
        'from': sender_address,
        'to': recipient_address,
        'value': amount,
        'gas': 21000,
        'gasPrice': Web3.to_wei('50', 'gwei'),
        'nonce': w3.eth.get_transaction_count(sender_address),
    }

    signed_transaction = w3.eth.account.sign_transaction(transaction, sender_private_key)

    transaction_hash = w3.eth.send_raw_transaction(signed_transaction.raw_transaction)

    print(transaction_hash)
```

Hình 10. Đoạn chương trình thực hiện chuyển tiền thông qua IPC kết nối với chuỗi khối Geth.

Tới đây, các em cần tạo lại 1 account alias kết hợp với 1 tài khoản trong mạng Geth, ở đây ta sẽ tạo 1 account alias đặt tên là `is355_` kết nối với tài khoản có địa chỉ em tạo ra lần 2 (hoặc sử dụng địa chỉ `sender_address` trong hình 10). Các em lấy thông tin private key địa chỉ này trên mạng Geth bằng 3 câu lệnh sau vào trong chương trình hình 10 và chạy lại. Thông tin tài khoản Passphrase các em tự đặt, và lưu ý các em sẽ nhớ tài khoản này để chuẩn bị thực hiện cho quá trình deploy.

```
transaction_hash = w3.eth.send_raw_transaction(signed_transaction.raw_transaction)

private_key_bytes = w3.eth.account.decrypt(encrypted_key, password)
# hiển thị hex (cẩn thận!)
private_key_hex = private_key_bytes.hex()
print("PRIVATE KEY (hex): 0x" + private_key_hex)

print(transaction_hash)
```

Hình 11. Câu lệnh lấy private key của một địa chỉ trong mạng Geth.

Lúc này mọi thứ đã có gần đầy đủ, bây giờ ta sẽ tạo thêm 1 file `deploy.py` trong thư mục scripts có nội dung như hình 12. Đây là đoạn chương trình gọi hàm `store` và `retrieve` trong smart contract các em đã viết ở hình 8.a.

```

from ape import accounts, project, networks

def main():
    dev = accounts.load("is355_")
    dev.set_autosign(True, passphrase="1234567890")

    # Deploy contract
    contract = project.SimpleStorage.deploy(sender=dev)
    print(f"Deployed at {contract.address}")

    # Gọi store() để lưu giá trị
    tx = contract.store(42, sender=dev)
    print("Stored value 42")

    # Gọi retrieve() bằng call() để đọc giá trị
    value = contract.retrieve.call()
    print(f"Retrieved value = {value}")

```

Hình 12. Đoạn chương trình tương tác với smart contract tự triển khai.

Sau khi hoàn tất, các em thực hiện compile bằng câu lệnh: `ape compile`

Và thực hiện triển khai smart contract và chạy chương trình bằng câu lệnh: `ape run deploy --network ethereum:local:http://127.0.0.1:8545`.

Sau khi thực hiện xong các em tạo 1 file `test_simple_storage.py` trong thư mục `test` có nội dung như hình 13.

```

import pytest

@pytest.fixture
def deployer(accounts):
    return accounts[0]

@pytest.fixture
def contract(deployer, project):
    return deployer.deploy(project.SimpleStorage)

def test_retrieve(contract):
    num = contract.retrieve.call()
    assert num == 0

def test_store(contract, deployer):
    contract.store(19, sender=deployer)
    num = contract.retrieve.call()
    assert num == 19

```

Hình 13. Nội dung chương trình `test_simple_storage.py`

Sau khi xong các em thực hiện câu lệnh: `ape test`. Các em chụp kết quả và thực hiện báo cáo, đồng thời giải thích các câu lệnh trong file `test_simple_storage.py`.

Ngoài 2 mạng blockchain được giới thiệu, còn có nhiều mạng blockchain thuộc platform Ethereum như: Sepolia, Infura, Alchemy, ... Các em có thể tìm hiểu để sử dụng trong bài tập đồ án môn học.

#### **4. Bài tập về nhà**

##### **Bài 1:** Kiểm tra “Sức khỏe” Blockchain

Yêu cầu: Viết chương trình Python kết nối đến node Geth và in ra:

1. Số block hiện tại
2. Thông tin block mới nhất (hash, miner, số giao dịch)
3. Gas trung bình của block đó.

Gợi ý: sử dụng `w3.eth.block_number`, `w3.eth.get_block()`.

##### **Bài 2:** Xây dựng Sổ Nợ Cá Nhân trên Blockchain

Yêu cầu:

1. Viết smart contract `DebtBook.vy` cho phép mỗi địa chỉ lưu trữ số tiền mà nó “nợ” một người khác.
2. Có hàm:
  - a. `addDebt(debtor: address, amount: uint256)` – chỉ owner được thêm.
  - b. `getDebt(debtor: address)` -> `uint256` – xem nợ của một người.
3. Deploy lên Geth và kiểm thử bằng Ape.

##### **Bài 3:** Thử nghiệm Giao dịch Thất bại

1. Viết một chương trình Python gửi giao dịch từ account A sang B với gas cố định đặt quá thấp.
2. Quan sát và phân tích lỗi mà Geth trả về.
3. Giải thích tại sao giao dịch không thành công và cách sửa.

##### **Bài 4:** Hệ thống Bỏ phiếu Phi tập trung

1. Viết smart contract `Voting.vy`:
  - a. Người tạo contract thêm các ứng cử viên.
  - b. Mỗi account chỉ được bỏ phiếu 1 lần.
  - c. Có hàm `getWinner()` để trả về ứng viên có nhiều phiếu nhất.
2. Deploy lên Geth.
3. Viết script Python cho:
  - a. 3 tài khoản khác nhau bỏ phiếu.
  - b. In ra kết quả bầu chọn cuối cùng

### **Bài 5:** Ứng dụng Blockchain trong Chuỗi Cung Ứng Nông Sản

Một hợp tác xã trồng rau sạch muốn chứng minh với khách hàng rằng sản phẩm của họ thật sự sạch và minh bạch.

- Vấn đề: Giấy chứng nhận dễ bị làm giả, khách hàng không tin hoàn toàn vào “tem xanh”.
- Giải pháp: Dùng blockchain để lưu lại hành trình của sản phẩm từ nông dân → kho → siêu thị. Mỗi bước được ghi nhận vào smart contract và không thể sửa.

Yêu cầu bài tập:

1. Viết smart contract SupplyChain.vy quản lý sản phẩm:
  - a. Struct Product gồm: id, name, currentOwner, status.
  - b. Hàm addProduct(id, name) → chỉ người quản lý thêm sản phẩm mới.
  - c. Hàm transferProduct(id, newOwner, newStatus) → cập nhật sản phẩm sang chủ sở hữu mới + trạng thái mới.
  - d. Hàm getProduct(id) → xem thông tin sản phẩm.
  - e. Event ProductUpdated log mọi lần cập nhật.
2. Triển khai contract lên mạng Geth bằng Ape.
3. Viết script Python mô phỏng:
  - a. Người quản lý thêm sản phẩm: "Rau cải sạch" ID = 1.
  - b. Chuyển từ nông dân → kho lạnh → siêu thị.
  - c. In thông tin sau mỗi bước để chứng minh minh bạch.
4. Mở rộng: Viết đoạn Web3.py query log event ProductUpdated để khách hàng kiểm tra lịch sử hành trình của sản phẩm.